

AWS

THE ANATOMY OF CLOUD INFRASTRUCTURE

Why Virtual Private Clouds, Network Isolation & Software-Defined Networking Are Non-Negotiable

TECHNICAL ARCHITECTURE BRIEF

Version 1.0 | Infrastructure & Security Division

EXECUTIVE SUMMARY

The internet is chaos. Billions of devices, millions of autonomous systems, and a fundamental design philosophy that was never intended to be secure — just survivable. The original architects of the TCP/IP stack were solving a Cold War problem: build a network that routes around damage. Security was an afterthought, trust was implicit, and isolation was practically non-existent.

Fast-forward fifty years and enterprises are building products on top of that same chaotic substrate. The question isn't whether you need cloud infrastructure — it's whether you understand exactly why every layer of that infrastructure exists, what happens when it's absent, and how it all fits together into a coherent, defensible architecture.

This document answers that question with precision. We'll dissect Amazon Web Services (AWS), peel apart Virtual Private Clouds (VPCs), understand network isolation at the packet level, and walk through how Software-Defined Networking (SDN) transforms hypervisor-level virtualization into programmable, policy-driven infrastructure.

BOTTOM LINE UP FRONT

Without VPCs, your compute instances are reachable from every other tenant on AWS. Without network isolation, a single misconfigured security group is a breach. Without SDN, your network is static, unauditable, and impossible to scale. Each layer in this stack is not optional — it's structural.

WHY AWS?

This question deserves a more rigorous answer than 'because everyone uses it.' Let's approach it from first principles: what does a production application actually need, and how do bare-metal alternatives compare?

The Physical Infrastructure Problem

Running servers on-premises sounds straightforward until you model the actual operational costs:

Concern	On-Premises Reality
Capital Expenditure	Hardware must be purchased 12–18 months in advance based on projected peak load. Over-provision and waste money; under-provision and fail during traffic spikes.
Power & Cooling	A single rack of modern servers consumes 10–20kW. Cooling systems must handle 1.2–1.4x that load. Data center power costs run \$0.03–\$0.12/kWh at scale.
Hardware Failure	MTBF (Mean Time Between Failures) for HDDs is ~1.5M hours. With 1,000 drives, expect one failure every 63 days — requiring hot spares, RAID, and 24/7 staffing.
Geographic Redundancy	Disaster recovery requires a secondary physical site with mirrored infrastructure, dedicated dark fiber links, and real-time replication — a multi-million dollar commitment.
Network Uplink	Enterprise-grade internet connectivity from multiple ISPs, BGP routing, and DDoS scrubbing capacity costs \$20,000–\$100,000/month before you've written a line of code.

AWS has already solved every one of these problems at a scale that no individual organization can replicate. Their power usage effectiveness (PUE) ratio approaches 1.2 — meaning only 20% overhead for cooling and infrastructure, compared to an industry average of 1.58. They negotiate bulk fiber contracts that cost individual enterprises 100x more per Gbps.

The Global Backbone

As of 2024, AWS operates 33 geographic Regions, each containing a minimum of three Availability Zones (AZs). Each AZ is a physically distinct data center facility with independent power, cooling, and networking — separated by meaningful physical distance (typically 60–100km) but connected by dedicated, low-latency fiber.

This is the AWS global backbone: a private, inter-region network that carries traffic between regions over AWS-owned fiber, completely bypassing the public internet. When your application

in us-east-1 replicates data to eu-west-1, that traffic travels through AWS infrastructure — not through Comcast or Deutsche Telekom.

LATENCY MATHEMATICS

The speed of light in fiber is approximately 200,000 km/s (light slows in glass by ~33%). New York to London is ~5,500 km, giving a theoretical minimum RTT of ~55ms. AWS achieves ~70ms region-to-region, which is near-theoretical for the physical distance. You cannot build this yourself without laying your own transatlantic cable.

■ The Economic Model: CapEx vs OpEx

AWS operates on a consumption model — you pay for what you use, measured in fractions of hours. The financial engineering implications are profound:

- Capital is not tied up in depreciating hardware assets. A \$500,000 server purchase becomes a \$0.50/hour compute charge, transforming a balance sheet liability into a variable operating expense.
- Elasticity becomes mathematically trivial. An e-commerce site that processes 1,000 orders/day during normal periods and 50,000/day during Black Friday does not need to provision for peak permanently. Auto Scaling Groups add and remove instances in minutes.
- The break-even analysis consistently favors AWS for workloads below ~\$10M/year in infrastructure spend, and remains competitive well beyond that for organizations without hyperscaler-level procurement leverage.

VIRTUAL PRIVATE CLOUDS: YOUR NETWORK IN SOFTWARE

A VPC is the most important construct in AWS. Before VPCs existed (in the EC2-Classic era, pre-2009), every instance launched on AWS shared a flat network address space. Your server's neighbors were unknown, uncontrolled, and reachable. A misconfigured instance could attempt to communicate directly with another customer's database. The blast radius of any mistake was the entire AWS public cloud.

VPCs solved this by applying the same network isolation that physical networks achieve through VLANs and dedicated hardware — but entirely in software, at the hypervisor layer, at global scale.

What a VPC Actually Is

A VPC is a logically isolated section of the AWS cloud where you define an IP address space (a CIDR block), create subnets, configure routing tables, and attach gateways. The word 'logically' is important — there is no dedicated physical hardware. Your VPC exists as a software overlay network atop AWS's physical infrastructure.

When you create a VPC with CIDR block 10.0.0.0/16, you're claiming 65,536 private IP addresses. You then carve those into subnets:

```
VPC: 10.0.0.0/16 (65,536 addresses)

Public Subnet: 10.0.1.0/24 — 251 usable IPs (AWS reserves 5)
Public Subnet: 10.0.2.0/24 — 251 usable IPs
Private Subnet: 10.0.10.0/24 — 251 usable IPs
Private Subnet: 10.0.11.0/24 — 251 usable IPs
Database Subnet: 10.0.20.0/24 — 251 usable IPs
Database Subnet: 10.0.21.0/24 — 251 usable IPs
```

Each pair of subnets is placed in different Availability Zones. This is not optional for production: if your single AZ has a power failure, you lose everything. With multi-AZ subnets and an Application Load Balancer spanning them, one AZ can go completely offline and your application continues serving traffic — automatically.

The Routing Table: The Brain of Your VPC

Every subnet has an associated routing table that dictates how traffic flows. This is where the 'public' vs 'private' distinction becomes concrete, not conceptual:

```
Public Subnet Route Table:
10.0.0.0/16 → local                (VPC internal traffic)
0.0.0.0/0   → igw-0a1b2c3d4e5f67890 (Internet Gateway)

Private Subnet Route Table:
10.0.0.0/16 → local                (VPC internal traffic)
0.0.0.0/0   → nat-0a1b2c3d4e5f67890 (NAT Gateway)

Database Subnet Route Table:
10.0.0.0/16 → local                (VPC internal traffic ONLY)
[no default route — no outbound internet access]
```

The database subnet has no route to the internet. None. A packet destined for any address outside 10.0.0.0/16 is simply dropped — not rejected, not firewalled, dropped at the routing layer. Even if an attacker somehow compromised your database credentials and attempted to exfiltrate data over the internet, the network topology itself prevents it.

Security Groups: Stateful Packet Filtering at the Instance Level

Security Groups are the final enforcement layer — stateful firewalls attached to each elastic network interface (ENI). 'Stateful' means that if you allow inbound TCP port 443, the return traffic (outbound ACK packets) is automatically permitted without a matching outbound rule. The kernel connection tracking table handles this at the hypervisor.

Layer	Mechanism & Behavior
VPC CIDR	Defines the private IP address space. Traffic cannot enter or exit without explicit routing + gateway configuration.
Subnet Routing	Routing tables determine reachability. No route = packet dropped. This is the first line of defense.
Network ACLs	Stateless, ordered rules at the subnet level. Both inbound and outbound rules must be configured. Evaluated before traffic reaches instances.
Security Groups	Stateful, allow-only rules at the ENI level. Default: deny all inbound, allow all outbound. Can reference other Security Groups as sources.
Instance IAM Role	Even within the network, API calls to AWS services require valid IAM credentials. Compromising the network doesn't grant AWS API access.

The Security Group reference feature is particularly powerful. Instead of specifying IP ranges for your application servers, you reference the security group itself:

```
Database Security Group Inbound Rules:
```

```
Port 5432 Source: sg-0abc123 (application-tier-sg) → ALLOW
Port 5432 Source: sg-0def456 (admin-bastion-sg) → ALLOW
```

```
# Any instance NOT in those security groups cannot reach
# port 5432, regardless of network position or IP address.
# This is identity-based, not address-based access control.
```

NETWORK ISOLATION: WHY FLAT NETWORKS ARE ATTACK SURFACES

Network isolation is not a compliance checkbox — it is a fundamental security control that limits the blast radius of any incident. To understand why isolation matters, you need to understand what happens without it.

■ The Lateral Movement Problem

Consider an unisolated network: a single flat layer-2 broadcast domain where every host can reach every other host. This is how many enterprise on-premises networks were built in the 1990s and 2000s. It's also how many cloud environments are accidentally configured today.

When a single internet-facing service is compromised — say, a web application with an unpatched SQL injection vulnerability — the attacker's next step is lateral movement. They have a foothold. They begin scanning:

```
# Attacker on compromised web server:
$ nmap -sn 10.0.0.0/16
> 10.0.0.1 - 10.0.255.254: ALL hosts reachable

$ nmap -p 5432,3306,6379,27017 10.0.0.0/16
> 10.0.20.5:5432 - PostgreSQL OPEN
> 10.0.20.6:3306 - MySQL OPEN
> 10.0.20.7:6379 - Redis OPEN (no auth by default)
```

In a properly isolated architecture, none of those ports are reachable from the web tier. The routing tables don't allow cross-tier communication. The security groups only permit the application tier to talk to the database tier on the specific port they need. The attacker has a compromised web server — and nothing else.

■ Defense in Depth: The Military Analogy

Medieval castle architects didn't build one wall. They built concentric walls with a moat, a drawbridge, a portcullis, a courtyard, and a keep — each layer requiring an attacker to defeat a distinct defense independently. A breach of the outer wall did not grant access to the keep.

Network isolation is the software equivalent. Each tier — public, application, data — is a distinct security zone with controlled ingress and egress. An attacker who breaches the outer perimeter (your load balancer) still faces the application tier's security group, then the database tier's routing restrictions, then database authentication, then application-level access controls.

THE PRINCIPLE OF LEAST PRIVILEGE — APPLIED TO NETWORKS

Every network path that doesn't exist is an attack vector that doesn't exist. Don't create routes your application doesn't need. Don't open ports your services don't use. The minimal surface area is the secure surface area.

VPC Peering and Transit Gateway: Controlled Connectivity

Isolation doesn't mean complete disconnection. Different VPCs often need to communicate — a shared services VPC hosting internal tooling, a data VPC with analytics infrastructure, a production VPC. The mechanism for this controlled connectivity is VPC Peering and AWS Transit Gateway.

VPC Peering creates a direct network route between two VPCs using AWS's internal backbone — traffic never hits the internet. The key property is non-transitivity: if VPC-A peers with VPC-B, and VPC-B peers with VPC-C, VPC-A cannot reach VPC-C through that path. This is a deliberate security property, not a limitation.

```
# VPC Peering: Non-Transitive by Design

VPC-A (10.1.0.0/16) ←—peering—→ VPC-B (10.2.0.0/16)
                        |
                        peering
                        |
                        VPC-C (10.3.0.0/16)

# VPC-A CANNOT reach VPC-C through VPC-B.
# Requires explicit VPC-A <-> VPC-C peering.
# This prevents accidental network sprawl.
```

AWS Transit Gateway solves hub-and-spoke topologies: a single gateway that multiple VPCs attach to, with route table policies that control exactly which VPCs can communicate with which. A 50-VPC architecture that would require 1,225 individual peering connections ($n*(n-1)/2$) becomes 50 attachments to a single Transit Gateway with a policy table.

SOFTWARE-DEFINED NETWORKING: THE CONTROL PLANE ABSTRACTED

This is where infrastructure gets genuinely fascinating. AWS doesn't run your virtual network on traditional networking hardware — Cisco routers, Juniper switches, physical VLANs. Everything you've read about so far — VPCs, subnets, routing tables, security groups — is implemented as software, running in a purpose-built distributed system called the Nitro Hypervisor and backed by AWS's Nitro system.

Traditional Networking vs. SDN: A Fundamental Architecture Shift

In traditional networking, the control plane (the logic that decides how traffic is routed) and the data plane (the hardware that actually forwards packets) are tightly coupled inside the same physical device. Your Cisco switch decides where to send a packet and sends it. Configuration changes require CLI access to individual devices. Scaling means physically adding hardware.

Property	Traditional Networking vs SDN
Control Plane	Distributed across individual devices, each running routing protocols (OSPF, BGP, STP) to build local routing tables. Centralized: a logically centralized controller maintains a global view and pushes forwarding rules to data plane devices.
Configuration	Device-by-device CLI (SSH into each switch/router, type commands). Change propagation is manual and error-prone. API-driven. A single API call can modify network behavior across thousands of nodes simultaneously.
Programmability	Limited to vendor-specific CLIs and SNMP. Automation requires vendor-specific tooling. Open APIs (OpenFlow, RESTCONF, NETCONF) allow arbitrary programming of forwarding behavior.
Failure Domain	Control plane failure (e.g., routing protocol flap) affects all traffic on the affected device and its neighbors. Control plane is replicated and distributed. Data plane continues forwarding based on last-known rules even if controller is unreachable.

How AWS Implements SDN: The Nitro System

AWS's Nitro System is their purpose-built hypervisor and hardware offload architecture. Understanding Nitro is understanding why AWS networking doesn't behave like the VMware vSphere environment your company runs on-premises.

Traditional hypervisors run networking in software on the host CPU — a "soft switch" (like Open vSwitch) in the Linux kernel processes every packet. This works but has two problems: it consumes host CPU cycles (stealing capacity from your workload) and it creates a software bottleneck for high-throughput workloads.

Nitro offloads networking to dedicated ASICs (Application-Specific Integrated Circuits) — custom silicon designed by AWS specifically for packet processing. The Nitro Card handles encryption, routing, and packet filtering in hardware, at line rate, without consuming any of your instance's vCPU.

Traditional Hypervisor Network Stack:

```
VM vNIC → Soft Switch (OVS) → Host NIC → Physical Network  
[Host CPU cycles consumed]
```

AWS Nitro Network Stack:

```
VM vNIC → Nitro Card (ASIC) → Physical Network  
[Zero host CPU consumed]  
[Hardware enforcement of Security Groups]  
[Hardware encryption for VPC traffic]
```

When you configure a Security Group rule in the AWS Console, that rule is compiled and pushed to the Nitro Card's forwarding tables — not to software running on the host kernel. Packet filtering happens at wire speed in silicon. A security group with 100 rules processes packets in nanoseconds.

Encapsulation: How Your Private IPs Stay Private

Here's a mystery: your EC2 instance has a private IP of 10.0.1.5. But AWS's physical network carries traffic between servers in the same data center, across racks, through spine switches. Those spine switches don't know about 10.0.1.5 — that's a private, per-VPC address that could exist in millions of other VPCs.

The answer is encapsulation. AWS uses a proprietary encapsulation protocol (similar in principle to VXLAN, but custom) to wrap your VPC packets inside physical network packets. The outer packet uses physical network addresses; the inner packet carries your VPC private IPs.

Physical Network Packet (what actually travels over hardware):

```
Outer Ethernet Header: Physical MAC addresses  
Outer IP Header:      Physical server IPs (not yours)  
Outer UDP Header:     Encapsulation protocol port  
  
  Inner IP Header: 10.0.1.5 → 10.0.10.8  
  Inner TCP Header: Port 5432  
  Your Payload: SQL query data
```

```
# The spine switch sees physical addresses only.  
# VPC isolation is enforced by the Nitro Card,  
# which knows which VPC ID owns which packets.
```

The Nitro Card at the destination server unwraps the outer packet, verifies the VPC ID matches, checks the Security Group rules, and delivers the inner packet to the destination VM's virtual NIC. If the VPC IDs don't match — which is how multi-tenancy works — the packet is silently dropped. Not rejected. Dropped. The sending VM receives no indication that the destination even exists.

BGP, ASNs, and the Routing of Your Traffic

When traffic must leave AWS — or enter it from the internet — it transits through AWS's border routers running BGP (Border Gateway Protocol). BGP is the routing protocol of the internet, the system that allows one Autonomous System (AS) to tell others 'I can reach these IP prefixes.'

AWS operates AS 16509 (Amazon.com), one of the largest Autonomous Systems on the internet by prefix count and traffic volume. When you assign an Elastic IP address to your instance, AWS advertises that prefix to their upstream providers. The entire internet learns, via BGP propagation, that traffic for your IP should go to AWS AS 16509.

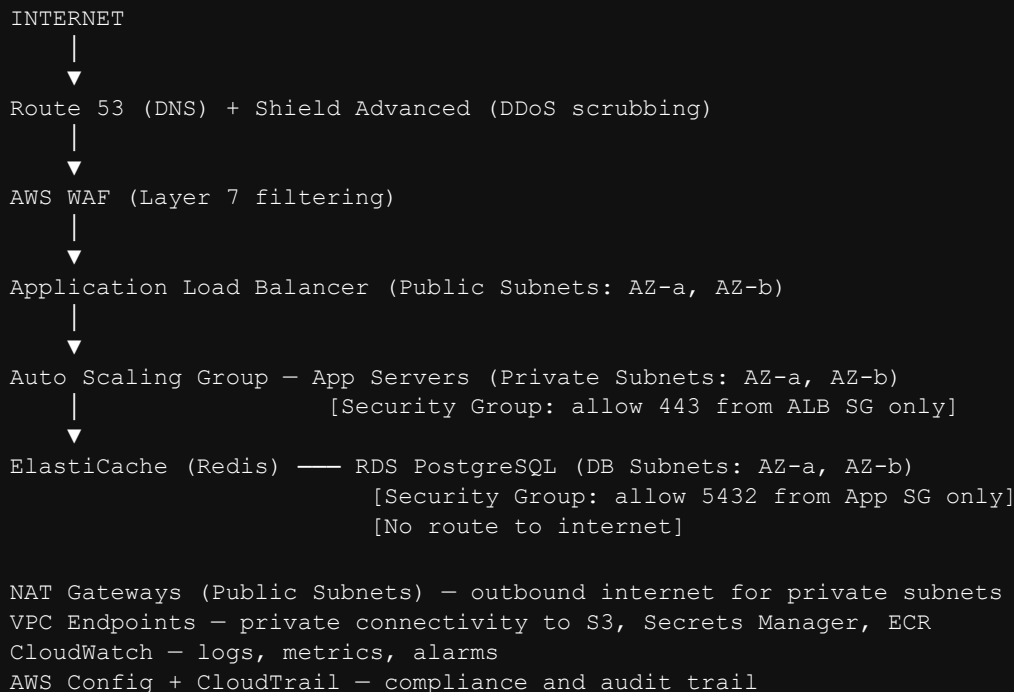
```
# BGP Advertisement (simplified):  
  
AWS AS 16509 → announces → 52.12.0.0/14  
               to peers:  
               NTT (AS 2914), Cogent (AS 174),  
               Telia (AS 1299), Level3 (AS 3356)  
  
# These tier-1 providers propagate the announcement globally.  
# Within seconds, routers worldwide know: packets for  
# 52.12.x.x belong to AWS.  
  
# AWS Direct Connect uses private BGP sessions on  
# dedicated fiber — your on-premises routers peer  
# directly with AWS, bypassing the public internet.
```

AWS Direct Connect is the logical extension: instead of routing your corporate traffic over the public internet to AWS, you establish a dedicated physical fiber connection from your data center (or colocation facility) to an AWS Direct Connect location. Your routers establish a private BGP session with AWS routers. Traffic travels on private infrastructure, with consistent latency and without internet-based jitter.

THE COMPLETE ARCHITECTURE: HOW IT ALL FITS

Let's construct a real-world production architecture and trace a single user request through every layer of this infrastructure — from their browser to your database and back.

The Reference Architecture



Packet Trace: A Single HTTPS Request

A user in Chennai hits your application. Here's what happens at every hop:

- DNS query for `api.yourapp.com` resolves via Route 53. Route 53 returns the Elastic IP of your ALB, selecting based on latency routing policy to the nearest AWS Region.
- TCP SYN arrives at the Internet Gateway, which performs NAT from the public Elastic IP to the ALB's private IP (e.g., 10.0.1.100). The IGW maintains a NAT translation table.
- The Network ACL on the public subnet evaluates the packet (stateless — checks both inbound and outbound rules). Port 443 is permitted.
- The ALB's Security Group checks: is this inbound TCP 443? Yes. Connection is allowed. TLS is terminated at the ALB — the backend never sees encrypted traffic from external clients.
- ALB selects a healthy target in the App tier Auto Scaling Group using its configured algorithm (round-robin, least-outstanding-requests). It opens a new connection to 10.0.10.25:8080 on a private subnet instance.

- The Nitro Card on the App server's host machine checks the Security Group: is this traffic from the ALB's Security Group on port 8080? Yes. Packet is delivered to the VM's vNIC.
- Your application code runs, queries Redis for cache, misses, queries RDS. The database connection goes to 10.0.20.5:5432. The Nitro Card on the RDS host checks: is this traffic from the App Security Group on port 5432? Yes. Connection is allowed.
- The database query returns results. Response travels back through the same path. Security Groups are stateful — no additional rules needed for return traffic. The ALB sends the HTTP response. TLS is re-encrypted. The user receives their data.

Every hop in that chain was governed by a software-defined policy that AWS enforces at the hardware level. None of it required a physical network change. Every rule is auditable, version-controlled (via Terraform/CloudFormation), and reviewable in IAM access logs.

■ Infrastructure as Code: The Final Argument

The most profound property of AWS SDN isn't performance or isolation — it's programmability. Your entire network topology can be expressed as code:

```
# Terraform - 3 lines to define a security group relationship
resource "aws_security_group_rule" "db_from_app" {
  type           = "ingress"
  from_port      = 5432
  to_port        = 5432
  protocol       = "tcp"
  source_security_group_id = aws_security_group.app.id
  security_group_id      = aws_security_group.database.id
}

# This code:
# - Is version-controlled in Git
# - Can be code-reviewed before deployment
# - Can be tested in a staging environment
# - Creates an immutable audit trail
# - Can be destroyed and recreated in minutes
```

A physical network doesn't offer this. You cannot git diff your Cisco switch configuration against last Tuesday's state and see exactly what changed. You cannot run terraform plan and see a preview of what will change before you execute. You cannot destroy your entire network and recreate it identically in another region in 10 minutes.

CONCLUSION: THE CASE IS STRUCTURAL

AWS is not a convenience play. It is a fundamental architectural choice that determines what's possible in your operations, security posture, and scaling model.

VPCs are not optional configuration — they are the isolation primitive without which multi-tenancy is a security catastrophe. Network isolation is not paranoia — it is the application of the blast radius principle to infrastructure, the same reason you use circuit breakers in code. SDN is not jargon — it is the mechanism by which all of this becomes programmable, auditable, and reproducible.

When you deploy infrastructure on AWS and understand these layers, you're not just running code in the cloud. You're operating a software-defined data center, with globally distributed compute, hardware-enforced isolation, and a network fabric that can be described, reviewed, and destroyed and recreated at will.

THE FINAL PRINCIPLE

The goal of all this infrastructure is not to be clever. It is to make the failure modes explicit, bounded, and recoverable. VPCs bound the blast radius. Network isolation bounds lateral movement. SDN makes the configuration auditable. Together, they transform infrastructure from a source of operational anxiety into a system with predictable, enforceable behavior.

END OF DOCUMENT

Infrastructure & Cloud Architecture Division | Confidential